

EJERCICIO DE APLICACIÓN

Colas sin prioridad y simulación del protocolo MQTT

1. Contexto del problema

En un sistema de Internet de las Cosas (IoT), varios dispositivos sensores envían mensajes a un servidor mediante el protocolo MQTT (Message Queuing Telemetry Transport). Los mensajes pueden llegar en diferentes momentos y deben ser procesados por el servidor.

Para esta actividad, se utilizará una cola sin prioridad, en la cual los mensajes se procesan siguiendo el principio FIFO (First In, First Out): el primer mensaje que ingresa es el primero que se retira.

El estudiante deberá implementar una simulación del funcionamiento de MQTT utilizando la especificación de cola disponible en el repositorio académico del docente:

<https://github.com/j-c-garciao/academics/tree/master/DataStructures/Queue>

Nota: el ejercicio se plantea sobre la interfaz conceptual de una cola FIFO y debe ajustarse a los nombres exactos de los métodos definidos en el repositorio.

2. Objetivo del ejercicio

Desarrollar un programa orientado a objetos que permita simular el envío y procesamiento de mensajes MQTT mediante una cola sin prioridad, aplicando la especificación de la estructura de datos Queue proporcionada por el docente.

Al finalizar la actividad, el estudiante deberá demostrar que puede:

- Utilizar una estructura de datos de tipo cola bajo el principio FIFO.
- Modelar mensajes de un sistema IoT mediante una clase.
- Simular la publicación de mensajes por parte de dispositivos sensores.
- Simular el procesamiento de mensajes por parte de un servidor MQTT.
- Identificar el comportamiento de la cola cuando se agregan y retiran elementos.

3. Enunciado del ejercicio

Una plataforma IoT recibe mensajes de tres dispositivos sensores instalados en una ciudad. Cada dispositivo envía mensajes al servidor MQTT. Para simplificar la simulación, el servidor almacenará los mensajes recibidos en una cola sin prioridad.

Dispositivo	Tipo de medición	Topic MQTT
S01	Temperatura	iot/sensor01/temperatura
S02	Humedad	iot/sensor02/humedad
S03	Nivel de agua	iot/sensor03/nivel

El sistema debe permitir simular dos operaciones principales:

- Publicación: un dispositivo genera un mensaje y lo agrega a la cola.
- Procesamiento: el servidor retira el siguiente mensaje de la cola y lo procesa.

Condición principal: el programa debe utilizar la especificación Queue del repositorio del docente. No se permite reemplazar la cola por un arreglo, una LinkedList utilizada directamente como cola o una estructura con prioridad.

4. Modelo de datos

Cada mensaje MQTT deberá representarse mediante una clase denominada MensajeMQTT.

Atributo	Tipo	Descripción
id	int	Identificador del mensaje
dispositivoid	String	Identificador del sensor
topic	String	Topic MQTT
payload	String	Valor de la medición
timestamp	String	Momento simulado de publicación

Ejemplo de mensaje:

```
ID: 1
Dispositivo: S01
Topic: iot/sensor01/temperatura
Payload: 28.5 °C
Timestamp: 10:00:01
```

5. Requerimientos funcionales

Requerimiento 1. Crear la cola de mensajes

Crear una instancia de la estructura Queue definida en el repositorio del docente, parametrizada para almacenar objetos de tipo MensajeMQTT, si la especificación lo permite. La cola debe comenzar vacía.

Requerimiento 2. Publicar un mensaje

Implementar una operación que reciba los datos de un mensaje MQTT, cree un objeto MensajeMQTT y lo agregue a la cola. La operación debe mostrar información del mensaje y confirmar su incorporación.

Requerimiento 3. Procesar el siguiente mensaje

Verificar si la cola contiene elementos, retirar el mensaje que se encuentra al frente, mostrar sus datos e indicar cuántos mensajes permanecen, si la especificación ofrece una operación de tamaño. Si la cola está vacía, mostrar un mensaje apropiado y evitar errores.

Requerimiento 4. Simular una secuencia de publicaciones

Ejecutar la secuencia de operaciones indicada en la sección siguiente y analizar el estado de la cola después de cada operación.

6. Secuencia de operaciones

Orden	Operación	Mensaje / acción
1	Publicar	S01 – Temperatura: 28.5 °C
2	Publicar	S02 – Humedad: 76 %
3	Publicar	S03 – Nivel: 45 cm
4	Procesar	Retirar el primer mensaje
5	Publicar	S01 – Temperatura: 29.1 °C
6	Procesar	Retirar el siguiente mensaje
7	Procesar	Retirar el siguiente mensaje

8	Procesar	Retirar el siguiente mensaje
9	Procesar	Intentar retirar de una cola vacía

7. Comportamiento esperado

Después de publicar los tres primeros mensajes:

Frente → Final: S01 → S02 → S03

Después de procesar S01:

Frente → Final: S02 → S03

Después de publicar nuevamente S01 con una temperatura de 29.1 °C:

Frente → Final: S02 → S03 → S01

Interpretación: el mensaje de S01 publicado en la última operación se agrega al final de la cola. No se procesa antes que los mensajes S02 y S03, aunque provenga de un dispositivo que ya había publicado anteriormente.

8. Diseño orientado a objetos sugerido

- MensajeMQTT: representa los datos del mensaje; incluye id, dispositivoId, topic, payload y timestamp.
- ServidorMQTT: gestiona la cola de mensajes e implementa las operaciones publicarMensaje() y procesarMensaje().
- Main o SimuladorMQTT: ejecuta la secuencia de prueba, crea mensajes y llama a las operaciones del servidor.

Esta es una propuesta de organización. El estudiante puede utilizar otros nombres de clases o distribuir las responsabilidades de manera diferente, siempre que mantenga el uso de la especificación de cola.

9. Restricciones de implementación

- Utilizar la especificación Queue suministrada por el docente.
- No utilizar PriorityQueue de Java.
- No ordenar los mensajes por dispositivo, valor de medición o tiempo.
- No eliminar directamente elementos intermedios de la cola.
- No utilizar estructuras de datos alternativas para realizar las operaciones principales de inserción y retiro.
- Mantener la lógica de publicación y procesamiento separada de la clase que representa el mensaje.
- Si la especificación de Queue requiere implementar una interfaz o extender una clase concreta, seguir el contrato establecido en el repositorio.

10. Preguntas de análisis

1. Después de publicar los tres primeros mensajes, ¿cuál es el orden de los elementos almacenados en la cola?

Respuesta: _____

2. ¿Por qué el mensaje S01 publicado en la operación 5 no se procesa inmediatamente?

Respuesta: _____

3. ¿Qué sucede cuando se intenta procesar un mensaje y la cola está vacía?

Respuesta: _____

4. ¿Qué operación de la cola permite garantizar el principio FIFO? Explique su respuesta con referencia a la especificación implementada.

Respuesta: _____

11. Entregables

- Código fuente del programa Java.
- Clase MensajeMQTT.
- Clase que gestione la simulación del servidor MQTT.
- Evidencia de ejecución de las nueve operaciones solicitadas.
- Respuestas a las cuatro preguntas de análisis.

12. Criterios de evaluación

Criterio	Porcentaje
Uso correcto de la especificación Queue	30 %
Implementación de la clase MensajeMQTT	15 %
Simulación de publicación y procesamiento FIFO	25 %
Manejo de cola vacía y validaciones	15 %
Diseño orientado a objetos y calidad del código	10 %
Respuestas de análisis	5 %
Total	100 %